

A Human-Readable Derivation of Fixed-Branch Zhao–Cui Tensor-Train Filtering

BayesFilter P13 Human-Readable Note

2026-05-31

Abstract

This note explains, from first principles, the part of the Zhao–Cui tensor-train sequential filtering construction that BayesFilter can use as a candidate high-dimensional nonlinear filtering method. The presentation is intended for a reader who knows probability and numerical linear algebra, but has not studied tensor-train filtering or the companion code. We start with ordinary filtering, work through a scalar nonlinear example, introduce tensor trains only after the filtering object is clear, and then state two propositions. The first proposition proves that the fixed-branch squared-TT recursion still produces normalized approximate filters. The second proves that its analytical gradient differentiates the same approximate likelihood scalar computed by the fixed branch. The result is a mathematical construction, not an accuracy theorem: it says exactly what is computed and how to differentiate it when the numerical branch is held fixed.

1 What Problem Are We Solving?

A state-space model describes a hidden state x_t , a possible static parameter ϑ , and an observed datum y_t . At time t , the filtering problem is to update the density of (x_t, ϑ) after seeing $y_1, \dots, y_t$. If the previous filter is known, the update is just Bayes’ rule, but in high dimension the integrals are the hard part.

Let

$$p_0(x_0, \vartheta), \quad f_\alpha(x_t \mid x_{t-1}, \vartheta), \quad g_\alpha(y_t \mid x_t, \vartheta)$$

be the prior, transition density, and observation density. The symbol α denotes an external model parameter whose likelihood derivative may be needed later. The symbol ϑ is a random coordinate carried by the filter.

Suppose the previous filtering density is

$$p_{t-1}(x_{t-1}, \vartheta \mid y_{1:t-1}).$$

Before seeing y_t , the model predicts

$$p(x_t, \vartheta \mid y_{1:t-1}) = \int f_\alpha(x_t \mid x_{t-1}, \vartheta) p_{t-1}(x_{t-1}, \vartheta \mid y_{1:t-1}) \, dx_{t-1}.$$

After observing y_t , Bayes’ rule multiplies this prediction by $g_\alpha(y_t \mid x_t, \vartheta)$ and divides by the normalizing constant.

It is useful to keep the old state inside the calculation. Define

$$u_t = (x_t, \vartheta, x_{t-1})$$

and

$$q_t(u_t; \alpha) = p_{t-1}(x_{t-1}, \vartheta \mid y_{1:t-1}) f_\alpha(x_t \mid x_{t-1}, \vartheta) g_\alpha(y_t \mid x_t, \vartheta).$$

This is an unnormalized joint density over current state, parameter, and old state. Its integral is

$$Z_t(\alpha) = \int q_t(x_t, \vartheta, x_{t-1}; \alpha) dx_t d\vartheta dx_{t-1}.$$

To see why this is the one-step evidence increment, integrate the joint density in the order x_t, ϑ, x_{t-1} :

$$\begin{aligned} Z_t(\alpha) &= \int g_\alpha(y_t \mid x_t, \vartheta) f_\alpha(x_t \mid x_{t-1}, \vartheta) p_{t-1}(x_{t-1}, \vartheta \mid y_{1:t-1}) dx_t d\vartheta dx_{t-1} \\ &= \int g_\alpha(y_t \mid x_t, \vartheta) p(x_t, \vartheta \mid y_{1:t-1}) dx_t d\vartheta = p_\alpha(y_t \mid y_{1:t-1}). \end{aligned}$$

The updated filter is the marginal of the normalized joint density:

$$p_t(x_t, \vartheta \mid y_{1:t}) = \int \frac{q_t(x_t, \vartheta, x_{t-1}; \alpha)}{Z_t(\alpha)} dx_{t-1}.$$

Thus a filtering algorithm needs three operations: approximate q_t , compute its integral, and marginalize out the old state. Zhao and Cui's TT method is one high-dimensional way to do exactly those three operations.

2 A Scalar Example Before Tensor Trains

Consider the one-dimensional nonlinear model

$$x_t = \rho x_{t-1} + \eta_t, \quad y_t = x_t^2 + \epsilon_t,$$

where $\eta_t \sim N(0, \sigma_\eta^2)$ and $\epsilon_t \sim N(0, \sigma_\epsilon^2)$. Suppose the previous approximate filter has density $\hat{p}_{t-1}(x_{t-1})$. The unnormalized two-variable object for one filtering step is

$$q_t(x_t, x_{t-1}) = \hat{p}_{t-1}(x_{t-1}) \frac{1}{\sigma_\eta} \varphi\left(\frac{x_t - \rho x_{t-1}}{\sigma_\eta}\right) \frac{1}{\sigma_\epsilon} \varphi\left(\frac{y_t - x_t^2}{\sigma_\epsilon}\right),$$

where φ is the standard normal density.

The observation $y_t = x_t^2 + \epsilon_t$ makes this a useful toy example. If y_t is positive and the noise is small, the likelihood is large near $x_t \approx \sqrt{y_t}$ and $x_t \approx -\sqrt{y_t}$. The posterior can be two-peaked even when the transition is Gaussian. A single Gaussian projection may miss that shape. A square-root density approximation tries to represent the non-Gaussian object more directly.

Let $\phi_t(x_t, x_{t-1})$ be an approximation to $\sqrt{q_t(x_t, x_{t-1})}$. The squared approximation is

$$\hat{q}_t(x_t, x_{t-1}) = \phi_t(x_t, x_{t-1})^2 + \tau_t \lambda_t(x_t, x_{t-1}),$$

where $\tau_t \geq 0$, $\lambda_t \geq 0$, and

$$\iint \lambda_t(x_t, x_{t-1}) dx_t dx_{t-1} = 1.$$

The added term is a small defensive density. It prevents the approximation from assigning exactly zero mass in places where the numerical square-root fit is unreliable.

The approximate evidence increment is

$$\widehat{Z}_t = \iint \widehat{q}_t(x_t, x_{t-1}) dx_t dx_{t-1} = \iint \phi_t(x_t, x_{t-1})^2 dx_t dx_{t-1} + \tau_t.$$

The approximate joint density after observing y_t is

$$\widehat{\pi}_t(x_t, x_{t-1}) = \frac{\widehat{q}_t(x_t, x_{t-1})}{\widehat{Z}_t}.$$

The next filter is its marginal over the old state:

$$\widehat{p}_t(x_t) = \int \widehat{\pi}_t(x_t, x_{t-1}) dx_{t-1}.$$

For a completely explicit defensive term, choose two normalized one-dimensional densities λ_x and λ_{x-} , and set

$$\lambda_t(x_t, x_{t-1}) = \lambda_x(x_t)\lambda_{x-}(x_{t-1}).$$

Then

$$\begin{aligned} \widehat{p}_t(x_t) &= \frac{1}{\widehat{Z}_t} \int \phi_t(x_t, x_{t-1})^2 dx_{t-1} + \frac{\tau_t}{\widehat{Z}_t} \lambda_x(x_t) \int \lambda_{x-}(x_{t-1}) dx_{t-1} \\ &= \frac{1}{\widehat{Z}_t} \int \phi_t(x_t, x_{t-1})^2 dx_{t-1} + \frac{\tau_t}{\widehat{Z}_t} \lambda_x(x_t). \end{aligned}$$

This line closes the scalar recursion: fit ϕ_t , add the defensive product density, normalize by $\widehat{Z}_t$, and integrate out the old state. If ϕ_t^2 is close to q_t , this filter should be a useful approximation. Even if it is not close, the algebra above still gives a nonnegative normalized density. That distinction matters: normalization is a mathematical property of the construction; accuracy is a separate numerical question.

3 From The Scalar Example To Tensor Trains

In the scalar example, ϕ_t is a function of two variables. In a high dimensional state-space model, $u_t = (x_t, \vartheta, x_{t-1})$ may have tens or hundreds of coordinates. A full tensor-product approximation over all coordinates is usually impossible because the number of coefficients grows exponentially.

A tensor train writes a high-dimensional function as a chain of smaller matrix-valued functions. Let $u = (u_1, \dots, u_D)$. A functional TT representation is

$$\phi(u) = G_1(u_1)G_2(u_2) \cdots G_D(u_D),$$

where

$$G_k(u_k) \in \mathbb{R}^{r_{k-1} \times r_k}, \quad r_0 = r_D = 1.$$

The integers r_k are TT ranks. Each G_k is a small matrix as a function of one coordinate. Multiplying the matrices yields a scalar.

For computation, each core is expanded in a one-dimensional basis:

$$G_k(u_k) = \sum_{a=1}^{n_k} C_{k,a} b_{k,a}(u_k), \quad C_{k,a} \in \mathbb{R}^{r_{k-1} \times r_k}.$$

If the ranks r_k stay moderate, the number of stored coefficients scales like $\sum_k r_{k-1} n_k r_k$, rather than $\prod_k n_k$. That is the basic high-dimensional promise: the representation is efficient when the target density has low-rank structure in a suitable coordinate order and basis.

The squared representation uses this TT for a square-root density. Once ϕ is represented by (3), the approximation $\phi^2 + \tau\lambda$ is automatically nonnegative. The integral of ϕ^2 can also be computed by contractions. If

$$M_k[a, b] = \int b_{k,a}(u_k) b_{k,b}(u_k) d\nu_k(u_k),$$

then define

$$R_D = 1, \quad R_{k-1} = \sum_{a,b=1}^{n_k} M_k[a, b] C_{k,a} R_k C_{k,b}^\top, \quad k = D, \dots, 1.$$

The scalar R_0 is exactly $\int \phi(u)^2 d\nu(u)$ for the declared cores and mass matrices. The approximation is in the fitted TT cores; the contraction itself is just algebra.

4 Zhao–Cui Algorithm In Human Language

The Zhao–Cui method applies the square-root TT idea sequentially. At each time t , it starts with the previous approximate filter, forms the joint object q_t from (1), fits a TT approximation to a square root of that object, squares the TT to obtain a nonnegative density, normalizes it, and marginalizes out x_{t-1} . Written in BayesFilter notation, the step is:

$$q_t(x_t, \vartheta, x_{t-1}) = \hat{p}_{t-1}(x_{t-1}, \vartheta) f_\alpha(x_t | x_{t-1}, \vartheta) g_\alpha(y_t | x_t, \vartheta).$$

The fitted square-root TT is

$$\sqrt{q_t(u_t)} \approx \phi_t(u_t) = G_{t,1}(u_{t,1}) \cdots G_{t,D}(u_{t,D}).$$

The nonnegative approximation is

$$\hat{q}_t(u_t) = \phi_t(u_t)^2 + \tau_t \lambda_t(u_t), \quad \int \lambda_t(u_t) du_t = 1.$$

The approximate normalizer is

$$\hat{Z}_t = \int \hat{q}_t(u_t) du_t,$$

and the next filter is

$$\hat{p}_t(x_t, \vartheta) = \int \frac{\hat{q}_t(x_t, \vartheta, x_{t-1})}{\hat{Z}_t} dx_{t-1}.$$

This is the filtering connection in one sentence: the TT is not replacing the state-space model; it is a deterministic representation used to approximate the joint density whose normalization and marginalization are the filtering update.

Conditional Knothe–Rosenblatt maps enter after the squared-TT density has been built. Because the squared TT can be integrated coordinate by coordinate, one can form marginal and conditional distributions and then sample by inverting conditional cumulative distribution functions. That transport machinery is important for sampling, smoothing, and sequential state-parameter learning, but the two propositions below only need the density, normalization, and marginalization pieces. Zhao and Cui present the sequential TT construction, squared-TT density, marginalization, and conditional transport machinery in their state-space formulation; Cui and Dolgov provide the underlying squared inverse Rosenblatt transport construction [1, 2].

5 Why The Published Adaptive Algorithm Is Not Yet A Gradient Algorithm

For filtering alone, adaptation is natural. A good TT implementation may change ranks, interpolation points, active subspaces, local approximation points, or truncation decisions as the data and parameter values change. These choices are part of how the method stays efficient.

A likelihood gradient has a stricter requirement. It must be the derivative of one scalar formula. If a small change in α changes the selected rank, pivot set, sample set, truncation, or linear algebra branch, then the formula being differentiated has changed. The derivative of yesterday's branch is still useful as a local sensitivity, but it is not an ordinary global derivative of the adaptive algorithm.

This does not make the adaptive method wrong. It only separates two uses:

- For filtering and sampling, adaptive TT construction may be the right numerical tool.
- For a same-scalar analytical gradient, we must either freeze the branch or design a smooth adaptation rule and differentiate that smooth rule.

The rest of this note follows the first option. We freeze the numerical branch so the value and derivative refer to the same approximate likelihood.

6 The Fixed-Branch Algorithm

A fixed branch is a fully specified deterministic version of one TT filtering path. It freezes every discrete or branch-changing choice that would otherwise make the scalar formula change while differentiating. The fixed branch does not claim to be more accurate than the adaptive algorithm. Its purpose is to make the approximate likelihood and its analytical gradient unambiguous.

At each time t , choose the following objects before differentiating:

1. the coordinate order for $u_t = (x_t, \vartheta, x_{t-1})$;
2. the reference domain or coordinate transform for each coordinate;
3. the one-dimensional bases $b_{k,a}$ and mass matrices M_k ;
4. the TT ranks $r_{t,k}$;
5. the interpolation, cross, or least-squares points used to fit the cores;
6. the local solver schedule, including sweep counts and stopping rules that are treated as fixed;
7. the defensive density λ_t and defensive mass τ_t ;
8. any stabilizing constant c_t used to shift the fitted potential;
9. the signs and active ranks for QR, SVD, Cholesky, or related factorizations.

After these choices are fixed, each step is an ordinary differentiable calculation, provided the model densities and linear solves are differentiable on that branch.

Forward step. Given $\hat{p}_{t-1}^{\text{TT}}$, form

$$q_t(u_t; \alpha) = \hat{p}_{t-1}^{\text{TT}}(x_{t-1}, \vartheta; \alpha) f_\alpha(x_t | x_{t-1}, \vartheta) g_\alpha(y_t | x_t, \vartheta).$$

If a stabilizing shift is used, write the square-root target evaluations as

$$s_t(u; \alpha) = \exp\left(-\frac{1}{2}[V_{t,0}(u; \alpha) - c_t(\alpha)]\right),$$

where $V_{t,0} = -\log q_t$. Equivalently, $s_t = \exp(c_t/2)\sqrt{q_t}$. Fit TT cores

$$\phi_t(u; \alpha) = G_{t,1}(u_1; \alpha) \cdots G_{t,D}(u_D; \alpha)$$

to these fixed target evaluations using the chosen interpolation or least-squares equations.

Compute the square mass $R_{t,0}$ by (3). Then

$$\hat{z}_t(\alpha) = R_{t,0}(\alpha) + \tau_t(\alpha)$$

for fixed normalized λ_t . The original-scale likelihood increment represented by the shifted fit is

$$\hat{L}_t^{\text{TT}}(\alpha) = \exp[-c_t(\alpha)] \hat{z}_t(\alpha).$$

The declared log-likelihood contribution is therefore

$$\hat{\ell}_t^{\text{TT}}(\alpha) = \log \hat{z}_t(\alpha) - c_t(\alpha).$$

Normalize and marginalize:

$$\hat{\pi}_t(u_t; \alpha) = \frac{\phi_t(u_t; \alpha)^2 + \tau_t(\alpha)\lambda_t(u_t)}{\hat{z}_t(\alpha)},$$

$$\hat{p}_t^{\text{TT}}(x_t, \vartheta; \alpha) = \int \hat{\pi}_t(x_t, \vartheta, x_{t-1}; \alpha) dx_{t-1}.$$

Pseudocode.

1. Input the previous normalized TT filter $\hat{p}_{t-1}^{\text{TT}}$, model densities f_α, g_α , datum y_t , and the frozen branch specification.

2. Build the target evaluation table. For each frozen fitting point $u^{(j)} = (x_t^{(j)}, \vartheta^{(j)}, x_{t-1}^{(j)})$, evaluate

$$q_t(u^{(j)}; \alpha) = \hat{p}_{t-1}^{\text{TT}}(x_{t-1}^{(j)}, \vartheta^{(j)}; \alpha) f_\alpha(x_t^{(j)} | x_{t-1}^{(j)}, \vartheta^{(j)}) g_\alpha(y_t | x_t^{(j)}, \vartheta^{(j)}).$$

3. Apply the frozen coordinate maps and stabilizing shift. Store $s_t(u^{(j)}; \alpha) = \exp(c_t/2)\sqrt{q_t(u^{(j)}; \alpha)}$, or the equivalent shifted square-root target used by the branch.

4. Solve for TT cores core-by-core. If the branch is interpolation-based, solve the frozen systems $A_{t,k}g_{t,k} = b_{t,k}$. If the branch is least-squares-based, solve the frozen normal equations $A_{t,k}^\top W_{t,k} A_{t,k}g_{t,k} = A_{t,k}^\top W_{t,k} b_{t,k}$. The branch must choose one of these value paths, or a precisely specified variant, before differentiation.

5. Assemble $\phi_t(u) = G_{t,1}(u_1) \cdots G_{t,D}(u_D)$. Contract the cores with the mass matrices to compute $R_{t,0}$.

6. Set $\hat{z}_t = R_{t,0} + \tau_t$ and store $\hat{\ell}_t^{\text{TT}} = \log \hat{z}_t - c_t$.

7. Define the normalized joint approximation

$$\hat{\pi}_t(u) = \{\phi_t(u)^2 + \tau_t \lambda_t(u)\} / \hat{z}_t.$$

8. Marginalize the old state to obtain

$$\hat{p}_t^{\text{TT}}(x_t, \vartheta) = \int \hat{\pi}_t(x_t, \vartheta, x_{t-1}) dx_{t-1}.$$

Store the marginal numerator $a_t(x_t, \vartheta)$ before division by $\hat{z}_t$, because the next derivative step needs $\partial_i \log \hat{p}_t^{\text{TT}}$.

9. Store the cores, fitting matrices $A_{t,k}$, right sides $b_{t,k}$, weights $W_{t,k}$ if used, target evaluations, coordinate maps, mass contractions $R_{t,k}$, marginal contractions, numerator a_t , shift c_t , defensive mass τ_t , and any factorization signs or active ranks consumed by the derivative pass.

10. Output $\hat{p}_t^{\text{TT}}$, $\hat{\ell}_t^{\text{TT}}$, and the saved branch data.

7 Why We Need Two Propositions

There are two different mathematical questions.

First, after replacing q_t by $\phi_t^2 + \tau_t \lambda_t$, do we still have a legitimate filtering recursion? A filter must be nonnegative, must integrate to one, and must be carried forward in time by prediction, update, normalization, and marginalization. Proposition 1 answers exactly that question for the fixed-branch approximation. It does not say the approximation is accurate; it says the approximation is a proper normalized filter.

Second, if we use

$$\hat{\ell}_T^{\text{TT}}(\alpha) = \sum_{t=1}^T \{\log \hat{z}_t(\alpha) - c_t(\alpha)\}$$

as a declared approximate likelihood, does the proposed gradient differentiate that same scalar? Proposition 2 answers that question. It does not differentiate the globally adaptive code; it differentiates the frozen branch whose forward pass produced the scalar.

8 Proposition 1: The Fixed Branch Still Filters

Plain-language claim. If the previous approximate filter is a normalized density, and if the fixed branch creates a nonnegative finite joint approximation $\hat{q}_t = \phi_t^2 + \tau_t \lambda_t$, then normalizing and marginalizing it gives another normalized density. Repeating this step gives a sequence of proper approximate filters.

Objects being approximated. The exact object is q_t in (1). The fixed branch uses

$$\hat{q}_t(u_t; \alpha) = \phi_t(u_t; \alpha)^2 + \tau_t(\alpha) \lambda_t(u_t)$$

instead. The filter is obtained from the normalized marginal of $\hat{q}_t$.

Assumption 1 (Dominated fixed-branch recursion). For each t , the previous approximate filter $\hat{p}_{t-1}^{\text{TT}}$ is nonnegative and integrates to one. The transition and observation densities are nonnegative and measurable. The fixed branch produces a measurable ϕ_t . The defensive density satisfies $\lambda_t \geq 0$ and $\int \lambda_t = 1$. The defensive mass satisfies $\tau_t \geq 0$. Finally,

$$0 < \hat{z}_t(\alpha) = \int \{\phi_t(u_t; \alpha)^2 + \tau_t(\alpha)\lambda_t(u_t)\} du_t < \infty.$$

Proposition 1 (Fixed-branch squared-TT recursion defines a normalized approximate filter). *Define*

$$\hat{\pi}_t(u_t; \alpha) = \frac{\phi_t(u_t; \alpha)^2 + \tau_t(\alpha)\lambda_t(u_t)}{\hat{z}_t(\alpha)}$$

and

$$\hat{p}_t^{\text{TT}}(x_t, \vartheta; \alpha) = \int \hat{\pi}_t(x_t, \vartheta, x_{t-1}; \alpha) dx_{t-1}.$$

Under the dominated fixed-branch assumptions, $\hat{p}_t^{\text{TT}}$ is a nonnegative normalized density. By induction from a normalized prior, the recursion defines a sequence of normalized approximate filtering densities.

Proof. The square ϕ_t^2 is nonnegative. The defensive term $\tau_t \lambda_t$ is also nonnegative because $\tau_t \geq 0$ and $\lambda_t \geq 0$. Therefore $\hat{q}_t = \phi_t^2 + \tau_t \lambda_t \geq 0$. The normalizer $\hat{z}_t$ is finite and positive by assumption, so $\hat{\pi}_t = \hat{q}_t / \hat{z}_t$ is well defined and nonnegative. Also

$$\int \hat{\pi}_t(u_t; \alpha) du_t = \frac{1}{\hat{z}_t(\alpha)} \int \hat{q}_t(u_t; \alpha) du_t = 1.$$

Now integrate the marginal:

$$\begin{aligned} \int \hat{p}_t^{\text{TT}}(x_t, \vartheta; \alpha) dx_t d\vartheta &= \int \int \hat{\pi}_t(x_t, \vartheta, x_{t-1}; \alpha) dx_{t-1} dx_t d\vartheta \\ &= \int \hat{\pi}_t(u_t; \alpha) du_t = 1. \end{aligned}$$

Tonelli's theorem justifies exchanging the order of integration because the integrand is nonnegative. Hence the new marginal is nonnegative and normalized. The prior gives the base case, and the same argument advances the filter from $t-1$ to t . Induction proves the sequence claim. $\square$

What this proposition gives us. It proves that the fixed-branch approximation is still a valid filtering recursion in the basic probabilistic sense: each output is a nonnegative normalized density.

What this proposition does not give us. It does not prove that $\hat{p}_t^{\text{TT}}$ is close to the exact filter. Exact recovery occurs only in the special case $\hat{q}_t = q_t$ almost everywhere at every time step.

9 Proposition 2: The Gradient Differentiates The Same Scalar

Plain-language claim. If the branch is fixed and all the displayed computations are differentiable, then differentiating the TT cores, mass contractions, defensive mass, and stabilizing constant gives the exact derivative of the scalar that the forward pass computed:

$$\hat{\ell}_T^{\text{TT}}(\alpha) = \sum_{t=1}^T \{\log \hat{z}_t(\alpha) - c_t(\alpha)\}.$$

Implementation path. The forward pass computes $\widehat{z}_t = R_{t,0} + \tau_t$, where $R_{t,0}$ is the mass contraction of ϕ_t^2 . The derivative pass must therefore differentiate the same contraction $R_{t,0}$, not a different approximation. It also must differentiate the previous filter because the next target $q_t = \widehat{p}_{t-1}^{\text{TT}} f g$ depends on that previous approximate filter. This is the practical meaning of “same scalar”: the backward pass must differentiate the stored forward branch that produced $\widehat{\ell}_T^{\text{TT}}$. If the derivative is compared against a fresh adaptive rerun with different ranks, pivots, samples, or factorization signs, the comparison is no longer testing the derivative of the same formula.

Assumption 2 (Fixed-branch differentiability). The assumptions of Proposition 1 hold. The branch choices are fixed. Along that branch, the model densities, previous approximate filter, coordinate maps, basis maps, TT core coefficients, defensive mass τ_t , and stabilizing constant c_t are differentiable in the parameter coordinate α_i . Differentiation may be interchanged with the finite contractions and the displayed integrals. Fixed interpolation or least-squares systems are nonsingular, or a specific differentiable pseudoinverse branch is declared.

Proposition 2 (Analytical gradient of the declared fixed-branch scalar). *Under the fixed-branch differentiability assumptions,*

$$\partial_i \widehat{\ell}_T^{\text{TT}}(\alpha) = \sum_{t=1}^T \left[\frac{\partial_i \widehat{z}_t(\alpha)}{\widehat{z}_t(\alpha)} - \partial_i c_t(\alpha) \right],$$

where, for fixed normalized λ_t ,

$$\partial_i \widehat{z}_t = 2 \int \phi_t(u; \alpha) \partial_i \phi_t(u; \alpha) du + \partial_i \tau_t.$$

If ϕ_t is represented by TT cores and $R_{t,0}$ is computed by the mass contraction (3), then the direct mass-contraction score is

$$\partial_i \widehat{\ell}_T^{\text{TT}} = \sum_{t=1}^T \left[\frac{\partial_i R_{t,0} + \partial_i \tau_t}{R_{t,0} + \tau_t} - \partial_i c_t \right].$$

Proof. Because the branch is fixed, the scalar is a finite sum of differentiable terms. For each t , $\widehat{z}_t > 0$, so

$$\partial_i \{\log \widehat{z}_t - c_t\} = \frac{\partial_i \widehat{z}_t}{\widehat{z}_t} - \partial_i c_t.$$

Summing over t gives (2).

Next,

$$\widehat{z}_t = \int \{\phi_t(u; \alpha)^2 + \tau_t(\alpha) \lambda_t(u)\} du.$$

The assumed interchange of differentiation and integration gives

$$\partial_i \widehat{z}_t = \int 2\phi_t(u; \alpha) \partial_i \phi_t(u; \alpha) du + \partial_i \tau_t \int \lambda_t(u) du.$$

Since $\int \lambda_t = 1$, this proves (2). If λ_t itself depends on α_i , an extra term $\tau_t \int \partial_i \lambda_t du$ appears; it vanishes only when the defensive density remains normalized on the differentiated branch.

It remains to connect $\partial_i \phi_t$ to the stored TT computation. If

$$\phi_t(u) = G_{t,1}(u_1) \cdots G_{t,D}(u_D),$$

then the product rule gives

$$\partial_i \phi_t(u) = \sum_{k=1}^D G_{t,1}(u_1) \cdots \partial_i G_{t,k}(u_k) \cdots G_{t,D}(u_D).$$

Using the basis expansion

$$G_{t,k}(u_k) = \sum_{a=1}^{n_k} C_{t,k,a} b_{k,a}(u_k),$$

the derivative $\partial_i G_{t,k}$ is obtained by replacing $C_{t,k,a}$ with $\partial_i C_{t,k,a}$, plus any basis-map derivative if the coordinate map or basis depends smoothly on α_i .

The derivative of the mass contraction follows by differentiating the same backward recursion that computed $R_{t,0}$. Starting from $\partial_i R_{t,D} = 0$,

$$\begin{aligned} \partial_i R_{t,k-1} = \sum_{a,b=1}^{n_k} M_k[a,b] & \left[(\partial_i C_{t,k,a}) R_{t,k} C_{t,k,b}^\top + C_{t,k,a} (\partial_i R_{t,k}) C_{t,k,b}^\top \right. \\ & \left. + C_{t,k,a} R_{t,k} (\partial_i C_{t,k,b})^\top \right]. \end{aligned}$$

This is the matrix product rule applied to the contraction formula (3). Therefore $\partial_i R_{t,0}$ is the derivative of the square mass used in the forward normalizer.

The core sensitivities come from the fixed core-construction equations. If a core vector $g_{t,k}$ is determined by a frozen interpolation equation

$$A_{t,k}(\alpha) g_{t,k}(\alpha) = b_{t,k}(\alpha),$$

then

$$A_{t,k} \partial_i g_{t,k} = \partial_i b_{t,k} - (\partial_i A_{t,k}) g_{t,k}.$$

If a core vector is determined by a fixed weighted least-squares problem

$$g_{t,k} = \arg \min_g \|W_{t,k}^{1/2} (A_{t,k} g - b_{t,k})\|_2^2$$

with fixed $W_{t,k}$ and nonsingular $N_{t,k} = A_{t,k}^\top W_{t,k} A_{t,k}$, differentiating the normal equations gives

$$\begin{aligned} N_{t,k} \partial_i g_{t,k} &= (\partial_i A_{t,k})^\top W_{t,k} (b_{t,k} - A_{t,k} g_{t,k}) \\ &+ A_{t,k}^\top W_{t,k} (\partial_i b_{t,k} - (\partial_i A_{t,k}) g_{t,k}). \end{aligned}$$

Thus the derivative pass solves the differentiated version of the same linear equations used by the forward pass.

Finally, the target values $b_{t,k}$ depend on the previous filter. From

$$q_t = \hat{p}_{t-1}^{\text{TT}} f_\alpha g_\alpha,$$

one has, wherever the factors are positive,

$$\partial_i \log q_t = \partial_i \log \hat{p}_{t-1}^{\text{TT}} + \partial_i \log f_\alpha + \partial_i \log g_\alpha.$$

If the previous filter can be written as

$$\hat{p}_{t-1}^{\text{TT}}(v; \alpha) = \frac{a_{t-1}(v; \alpha)}{\hat{z}_{t-1}(\alpha)},$$

then

$$\partial_i \log \hat{p}_{t-1}^{\text{TT}}(v; \alpha) = \frac{\partial_i a_{t-1}(v; \alpha)}{a_{t-1}(v; \alpha)} - \frac{\partial_i \hat{z}_{t-1}(\alpha)}{\hat{z}_{t-1}(\alpha)}.$$

The numerator a_{t-1} is a marginal contraction of the previous squared-TT object. Its derivative is obtained by the same marginal contraction with one previous core at a time replaced by its differentiated core, plus the derivative of τ_{t-1} and any defensive-density term. Therefore the sensitivity recursion follows the same time recursion as the filter itself.

Combining the log-normalizer derivative with the differentiated contraction gives the boxed score (2). Every term in that formula comes from differentiating the forward scalar $\sum_t \{\log \hat{z}_t - c_t\}$, so the gradient is a same-scalar gradient for the fixed branch. $\square$

What this proposition gives us. It gives an implementable analytical derivative for the declared fixed-branch approximate likelihood. A prototype can compute the forward scalar, save the branch data, compute core sensitivities, propagate mass-contraction sensitivities, propagate previous-filter sensitivities, and assemble (2).

What this proposition does not give us. It does not give a global derivative of an adaptive algorithm whose ranks, pivots, truncations, sample sets, or factorization branches change with α . It also does not prove that this approximate likelihood is close to the exact likelihood.

10 How To Compute The Gradient

The following recipe is the minimal implementation contract for the fixed-branch gradient.

Inputs. The implementation needs the state-space model, data, prior, frozen branch specification, defensive densities, defensive masses, basis and mass matrices, and derivatives of f_α , g_α , coordinate maps, shifts c_t , and any parameter-dependent branch-smooth objects.

Forward pass. For $t = 1, \dots, T$:

1. build the target evaluations of $q_t = \hat{p}_{t-1}^{\text{TT}} f_\alpha g_\alpha$;
2. fit the square-root TT cores using the frozen equations;
3. compute $R_{t,0}$, $\hat{z}_t = R_{t,0} + \tau_t$, and $\hat{\ell}_t^{\text{TT}} = \log \hat{z}_t - c_t$;
4. normalize and marginalize to obtain $\hat{p}_t^{\text{TT}}$;
5. save all contractions and solve matrices needed by the derivative pass.

Sensitivity pass. For each parameter coordinate α_i :

1. differentiate the target evaluations using the displayed log-target identity $\partial_i \log q_t = \partial_i \log \hat{p}_{t-1}^{\text{TT}} + \partial_i \log f_\alpha + \partial_i \log g_\alpha$.
2. solve the displayed interpolation sensitivity equation $A_{t,k} \partial_i g_{t,k} = \partial_i b_{t,k} - (\partial_i A_{t,k}) g_{t,k}$, or the displayed weighted least-squares sensitivity equation based on $N_{t,k} = A_{t,k}^\top W_{t,k} A_{t,k}$.

3. compute $\partial_i R_{t,0}$ by differentiating the same backward mass-contraction recursion used to compute $R_{t,0}$, replacing one core factor at a time by its differentiated core and propagating $\partial_i R_{t,k}$ backward.
4. compute $\partial_i \widehat{z}_t = \partial_i R_{t,0} + \partial_i \tau_t$;
5. compute the step score

$$\partial_i \widehat{\ell}_t^{\text{TT}} = \frac{\partial_i \widehat{z}_t}{\widehat{z}_t} - \partial_i c_t;$$

6. differentiate the normalized marginal filter using

$$\partial_i(\widehat{q}_t/\widehat{z}_t) = \frac{\partial_i \widehat{q}_t}{\widehat{z}_t} - \frac{\widehat{q}_t}{\widehat{z}_t^2} \partial_i \widehat{z}_t,$$

then marginalize to carry $\partial_i \widehat{p}_t^{\text{TT}}$ forward.

Parity test before using the gradient. For each coordinate, freeze the same branch and compare

$$\frac{\widehat{\ell}_T^{\text{TT}}(\alpha + h e_i) - \widehat{\ell}_T^{\text{TT}}(\alpha - h e_i)}{2h} \quad \text{with} \quad \partial_i \widehat{\ell}_T^{\text{TT}}(\alpha).$$

If the finite-difference calls change a rank, pivot, sample set, sign, truncation, or selected shift rule, the test has detected branch instability. That is not a proof that TT filtering fails; it is a warning that the frozen branch gradient no longer matches the adaptive value path being tested.

11 What This Construction Gives Us

The construction gives a clear approximate filtering object. At each time, it represents the unnormalized filtering joint density by a nonnegative squared-TT density, computes its normalizer, and marginalizes the old state. This is enough to produce a sequence of normalized approximate filters.

It also gives a declared approximate likelihood:

$$\widehat{\ell}_T^{\text{TT}} = \sum_t \{\log \widehat{z}_t - c_t\}.$$

For a fixed branch, the gradient in Proposition 2 is the derivative of that same scalar. This is the mathematical prerequisite for treating the method as a future gradient-based likelihood candidate: one knows which scalar is being differentiated and which operations must be differentiated.

The construction also gives a practical bridge from the published adaptive method to an implementable prototype. One can first implement the frozen branch, check value-gradient parity, and then study how often the adaptive branch changes in realistic filtering problems.

12 What Still Must Be Tested

Several important questions remain numerical rather than algebraic.

- Approximation accuracy: the propositions do not say that $\phi_t^2 + \tau_t \lambda_t$ is close to q_t .
- Branch stability: finite-difference perturbations may change ranks, pivots, sample points, truncations, or factorization signs.

- Conditioning: interpolation and least-squares systems can be ill conditioned, making both values and derivatives unstable.
- Scaling: the method is persuasive for high dimension only if the TT ranks remain moderate on the target model class.
- Comparison with fixed sparse-grid filtering: TT filtering is attractive when density or transport structure has stable low TT rank; sparse-grid filtering is attractive when the effective quadrature dimension is small.

These are not reasons to discard the method. They are the experiments and diagnostics needed before claiming that this approximation is reliable on a specific high-dimensional nonlinear state-space model.

A Source And Code Anchors

The main text derives the filtering objects before citing them. The source anchors below record where the same technical objects appear in the inspected literature and audit snapshot.

- Zhao–Cui equations (1)–(3) define the state-space transition, observation, and parameter notation used by their sequential learning problem [1].
- Zhao–Cui equations (9)–(12) and Algorithm 1 form the nonseparable sequential density over (x_t, θ, x_{t-1}) , approximate it in TT form, and integrate it.
- Zhao–Cui equation (13) and nearby Lemma 1 introduce the squared-TT defensive density of the form $\phi^2 + \tau\lambda$.
- Zhao–Cui Algorithm 2 applies the squared-TT construction sequentially.
- Zhao–Cui Section 4.1 discusses the exact joint normalizer as evidence.
- Zhao–Cui Propositions 2 and 4 support the marginal and conditional constructions used for filtering and conditional KR maps.
- Cui–Dolgov Sections 2–3 provide the inverse Rosenblatt, functional TT, and squared inverse Rosenblatt transport substrate.
- The audit snapshot of the companion code shows the same scalar path. In `models/full_sol.m`, the running marginal likelihood receives the increment `log(sirt.z)-const`. In the TT marginalization routine, the stored normalizer is `obj.z = obj.fun_z + obj.tau`. These code references show that the inspected MATLAB snapshot computes the same scalar structure; the mathematical justification remains the derivations in the main text.

B A Compact Checklist For A Prototype

Item	What must be specified
Model	Prior, transition, observation density, data, parameter coordinate, and all dimensions.
Branch	Coordinate order, domains, bases, ranks, fitting points, solver schedule, defensive density, defensive mass, and shift rule.
Forward scalar	$R_{t,0}$, $\hat{z}_t = R_{t,0} + \tau_t$, and $\hat{\ell}_t^{\text{TT}} = \log \hat{z}_t - c_t$.
Filter output	Normalized joint density and marginal $\hat{p}_t^{\text{TT}}(x_t, \vartheta)$.
Derivative	Target sensitivities, core sensitivities, contraction sensitivities, previous-filter sensitivities, $\partial_i \tau_t$, and $\partial_i c_t$.
Diagnostics	Positive finite normalizers, stable branch objects, conditioning, rank growth, and same-scalar finite-difference parity.

References

- [1] Zhao, Y. and Cui, T. (2024). Tensor-train methods for sequential state and parameter learning in state-space models. *Journal of Machine Learning Research*, 25(244):1–51.
- [2] Cui, T. and Dolgov, S. (2022). Deep composition of tensor-trains using squared inverse Rosenblatt transports. *Foundations of Computational Mathematics*, 22:1863–1922. DOI: 10.1007/s10208-021-09537-5.